



Wing-M130 Integration Manual

Version r1p0, 2024.07.15

Wingsemi Technology Wing-M130 Processor

Copyright © 2023-2025 Wingsemi Technology Co., Ltd. All rights reserved.

This document and all its contents, including but not limited to text, images, graphics, and code, are the exclusive property of Wingsemi Technology Co., Ltd. ("Wingsemi"). The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of this document may be reproduced, distributed, modified, or otherwise used without the express written permission of Wingsemi.

This document is Confidential. This document may only be used and distributed in accordance with the terms of the agreement from Wingsemi Technology.

目录

- 目录 II
- 修订记录 4
- 1. 整体介绍 1
 - 1.1. 内核介绍 1
 - 1.2. 关键特性列表 2
- 2. 配置说明 5
 - 2.1. 参数列表 5
 - 2.2. 典型配置 9
- 3. 接口信号描述 12
 - 3.1. 时钟与复位信号 12
 - 3.2. 调试接口信号 13
 - 3.3. 中断接口信号 13
 - 3.4. 外部数据接口信号(Master) 14
 - 3.5. 外部数据接口信号(Slave) 18
 - 3.6. 其他接口信号 19
 - 3.7. 低功耗接口 21
 - 3.8. Memory Wrap 接口信号 22
 - 3.9. DFT 接口信号 24
- 4. 关键特性集成说明 25
 - 4.1. 时钟与复位 25
 - 4.1.1. 时钟系统集成 25
 - 4.1.2. 复位系统集成 25
 - 4.1.3. 内核 Boot 流程 27
 - 4.2. 静态输入信号 28
 - 4.3. 主数据通路接口 29
 - 4.3.1. Master 类接口 29
 - 4.3.2. Slave 类接口 29
 - 4.3.3. 核内请求与 Master 接口路由关系 30
 - 4.3.4. Master 接口空间划分 30
 - 4.4. Memory Map 空间介绍 30
 - 4.4.1. 系统 Memory Map 空间介绍 30
 - 4.4.2. Wing-M130 核内 Memory Map 空间介绍 31
 - 4.5. PMA 配置说明 33

4.6.	存储子系统集成说明.....	34
4.6.1.	TCM 集成说明	34
4.6.2.	ICache 集成说明.....	34
4.6.3.	Memory Wrapper 集成说明.....	34
4.6.4.	Memory ECC 实现说明.....	35
4.7.	中断系统集成说明	35
4.7.1.	外部中断集成说明.....	35
4.7.2.	软件中断集成说明.....	36
4.7.3.	定时中断集成说明.....	36
4.7.4.	NMI 集成说明	36
4.7.5.	中断属性配置说明.....	36
4.8.	调试系统集成说明	37
4.8.1.	调试接口跨时钟域说明.....	37
4.9.	低功耗集成说明.....	38
4.9.1.	WFI 低功耗流程	38
4.10.	DFT 集成说明	39
5.	后端实现指导	40
5.1.	接口信号时序约束建议	40
5.2.	版图规划建议	40
6.	Wing-M130 HDK.....	41
6.1.	Wing-M130 directory	41
6.2.	HDK 使用说明	42
6.2.1.	Database 设置	42
6.2.2.	工具链依赖检查	42
6.2.3.	仿真环境说明.....	42
6.2.4.	Lint 检查环境说明	44
6.2.5.	综合环境使用说明.....	44
6.2.6.	JTAG 仿真环境使用说明	45
6.2.7.	Database 清除	45

修订记录

日期	版本	描述
2023/10/18	r0p0	Initial Version
2024/07/15	r1p0	Update for final release

1. 整体介绍

Wing-M130 是隼瞻科技有限公司开发的一款基于 RISC-V 的顺序三级流水处理器，具备低功耗，低成本，高代码密度等特点，适用于微控制器，IoT，传感器等对于成本/功耗/代码密度敏感领域。

1.1. 内核介绍

Wing-M130 是一款典型 Load-Store 架构的 RISC-V 处理器，运算指令只操作核内通用寄存器，通过 Load/Store 指令交换通用寄存器与内存中的值。Wing-M130 支持最大 32bit 寻址空间，默认支持小端存储。执行环境可以定义地址空间中哪些部分可以进行合法访问。

Wing-M130 的顶层架构图如下：

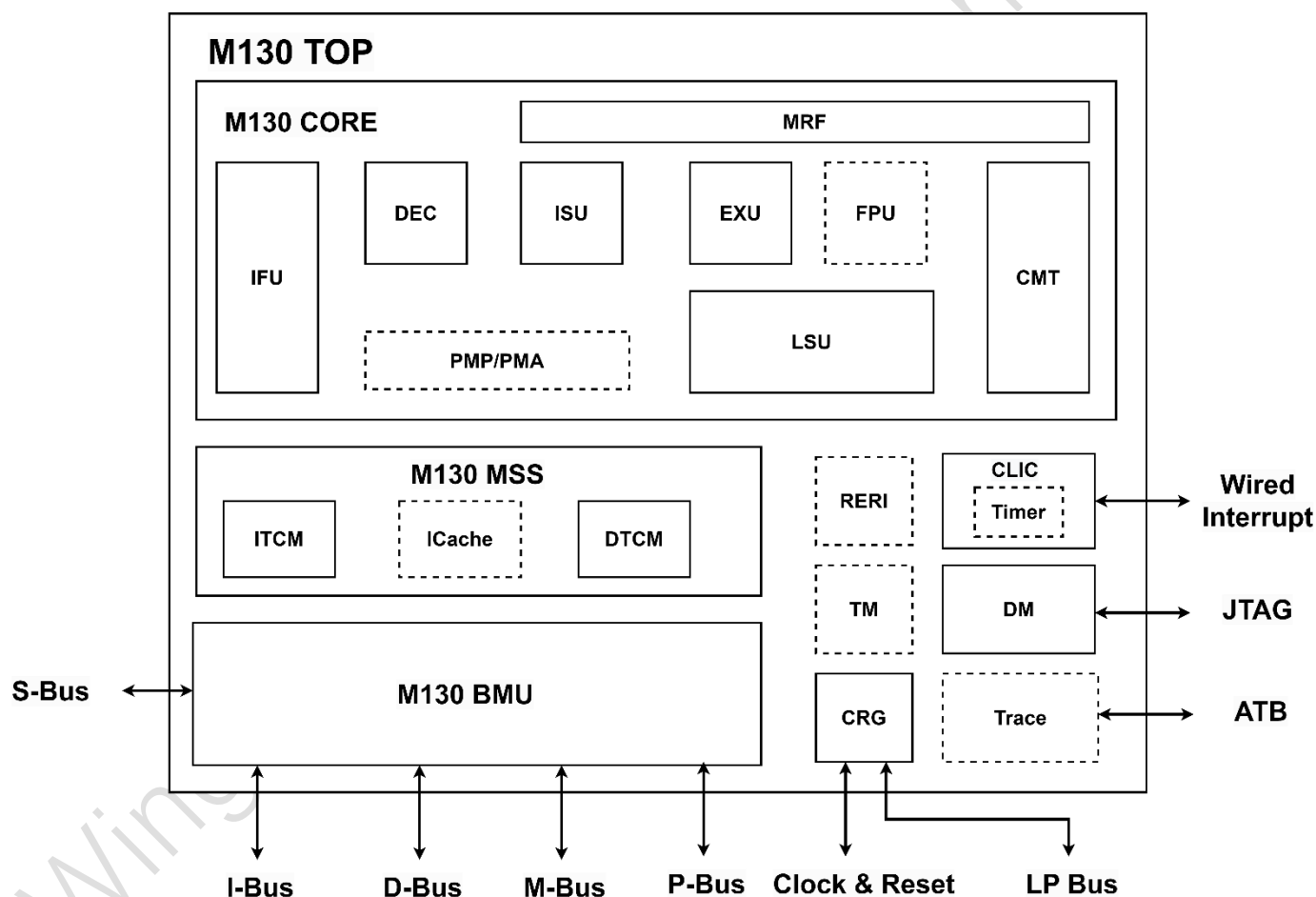


Figure 1-1 Wing-M130 顶层框图

Wing-M130 包含如下组件：

- Wing-M130 Core
 - Instruction Fetch Unit (IFU)

- Pipeline Control Unit (PCU)
 - Decoder
 - Main Register File
 - Issue Unit
 - Commit Unit
- Execution Unit (EXU) with
 - Integer Arithmetic Unit (IALU)
 - Optional Multiplication and Division Unit (MDU)
- Optional Floating Point Unit(FPU)
- Load-Store Unit (LSU)
- Optional PMP & PMA
- Memory Subsystem with
 - Instruction TCM
 - Data TCM
 - Optional Instruction Cache
- Interrupt Controller (CLIC or CLINT) with
 - Optional Real-Time Timer
- Debug System
 - DTM
 - DMI
 - DM
- Optional Trigger Module
- Clock and Reset Generation (CRG)
- Optional Trace Module
- Optional RAS Error Record Register Interface (RERI)
- Bus Matrix Unit
 - 32bit I-BUS AHB-Lite master interface
 - 32bit D-BUS AHB-Lite master interface
 - 32bit M-BUS AHB-Lite master interface
 - 32bit P-BUS APB master interface
 - 32bit S-BUS AHB-Lite slave interface

1.2. 关键特性列表

Wing-M130 的主要特性列表如下：

- 支持如下指令集：
 - RV32I 基础指令集（包含 32 个通用寄存器）或 RV32E 基础指令集（包含 16 个通用寄存器）
 - Zfencei 扩展指令集
 - Zicsr 扩展指令集
 - C/ZC 扩展指令集，包含 16bit 压缩指令
 - C 对应 ZC 中 zca 扩展
 - M 扩展指令集，包含乘法/除法扩展指令
 - 支持 ZMMUL 子扩展指令集，只包含乘法指令
 - B 扩展指令集
 - A 扩展指令集
 - Zicond 扩展指令集
 - XC/XB 扩展指令集（WingSemi 自定义扩展指令集）
- 支持 Machine 和 User 两个特权模式，其中 User 特权模式可配置
- 支持 3 级顺序流水线，IF，EX 和 WB stage
- 支持静态分支预测以及 RAS 返回地址预测
- 支持非对齐 Load/Store 操作（Non-Idempotent 和 Atomic 等操作除外）
- 支持配置快速硬件乘法实现或最多 33 个 cycle 的迭代乘法实现
- 支持最多 33 个 cycle 的迭代除法实现
- 支持可选的 PMP
 - 支持默认 8 个 PMP entry
 - 支持 NAPOT 和 TOR 地址匹配模式，默认仅支持 NAPOT 模式
 - 支持默认 32B PMP 地址配置粒度
- 支持可选的 PMA
 - PMA 与硬件平台绑定，运行中不可改，定义了平台硬件属性，包含 Cacheable 和 Idempotent 属性
 - 支持 NAPOT 和 TOR 地址匹配模式
- 支持可选的 Instruction TCM
- 支持可选的 Data TCM
- 支持可选的 Instruction Cache
- 支持四个独立对外访问 SoC 的 Master 接口
 - 位宽 32bit 的 AHB-Lite I-Bus 接口
 - 位宽 32bit 的 AHB-Lite D-Bus 接口
 - 位宽 32bit 的 AHB-Lite M-Bus 接口
 - 位宽 32bit 的 APB P-Bus 接口
 - 上述接口可以独立配置是否使能
 - 上述接口可以独立配置对应访问的地址范围

- 支持一个可配置的 32bit AHB-Lite 的 Slave 接口，S-Bus，允许 SoC 通过这个接口访问 Wing-M130 内部的寄存器空间或 TCM 空间
- 支持可配置的中断控制器，支持配置两种不同模式的中断控制器
 - 低成本基础的 CLINT 中断控制器，或
 - 低延迟，功能更强大的 CLIC 中断控制器
 - CLIC 支持配置最多 128 个外部中断
- 支持可配置的 RISC-V 调试系统
 - 调试系统包含：DM 调试模块，TM 断点模块以及 JTAG 调试接口
 - 支持 RISC-V 调试定义的 abstract command，access memory，quick access 和 Program Buffer
 - 支持调试系统 halt/resume 核，复位/解复位内核或整个平台等功能
 - 支持默认 4 个硬件断点配置
- 支持可配置的 N-Trace 功能
 - 仅支持指令 Trace 功能，产生的 Trace Message 存储在内部的 SRAM 中，可以通过调试器或外部总线访问
- 支持 3 个内嵌的 PMU 计数器
 - Real time clock
 - Cycle counter
 - Instructions-retired counter
 - 默认位宽均为 64bit
- 支持 RAS 特性
 - 支持可选的针对 Memory 的 ECC 保护（当配置了 Memory）
 - 支持 RAS 错误上报机制
- 支持基于 WFI 的 sleeping 和 deep sleeping 低功耗模式

2. 配置说明

本章节主要描述 Wing_M130 主要支持的可配置项，支持用户基于不同的功能需求/性能需求/PPA 需求配置 Wing-M130。

Wing-M130 采取宏定义的方式定义相关参数，代码会自动 include define 文件和 undefine 文件。

2.1. 参数列表

下表描述了所有 Wing-M130 的主要参数选项：

备注：下表中的参数名隐藏了统一的前缀 WING_M130，在配置/查看参数时需要用户关联相关前缀

Table 2-1 Wing-M130 主要配置参数表

Parameter Name	Type	Range	Default	Description
ISA 类参数				
RV32E_EXT	架构类	ON/OFF	OFF	当配置为 OFF 时，支持 RV32I 基础指令集，对应 32 个通用寄存器 当配置为 ON 时，支持 RV32E 基础指令集，对应 16 个通用寄存器
A_EXT	架构类	ON/OFF	OFF	当配置为 OFF 时，不支持 A 扩展指令集 当配置为 ON 时，支持 A 扩展指令集
B_EXT	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 B 扩展指令集 当配置为 ON 时，支持 B 扩展指令集 注：B 扩展指令集包含 zba zbb zbc zbs 四个子扩展集
C_EXT	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 C 扩展指令集 当配置为 ON 时，支持 C 扩展指令集
ZC_EXT	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 ZC 扩展指令集 当配置为 ON 时，支持 ZC 扩展指令集 注：ZC 扩展指令集包含 zca zcb zcmt zcmp 四个子扩展集，其中 zca 就是 C 扩展
M_EXT	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 M 扩展指令集 当配置为 ON 时，支持 M 扩展指令集
ZFINX_EXT	架构类	ON/OFF	OFF	当配置为 OFF 时，不支持 ZFINX 扩展指令集 当配置为 ON 时，支持 ZFINX 扩展指令集
ZICOND_EXT	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 ZICOND 扩展指令集

				当配置为 ON 时，支持 ZICOND 扩展指令集
WICE_XC_EXT	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 WingSemi 自定义 XC 扩展指令集 当配置为 ON 时，支持 WingSemi 自定义 XC 扩展指令集
SMCLIC_EXT	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 SMCLIC 扩展，此时中断控制器为 CLINT 当配置为 ON 时，支持 SMCLIC 扩展，此时中断控制器为 CLIC
SMCLICSHV_EXT	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 SMCLICSHV 扩展，不支持 CLIC 模式下的向量中断 当配置为 ON 时，支持 SMCLICSHV 扩展，支持 CLIC 模式下的向量中断 <i>注：这个参数配置为 ON 时，仅当 SMCLIC_EXT 也配置为 ON 时生效</i>
SDEXT_EXT	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 SDEXT 扩展，对应不支持调试相关的功能与组件（包含 DM/DTM/DMI） 当配置为 ON 时，支持 SDEXT 扩展，支持调试相关组件
SDTRIG_EXT	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 SDTRIG 扩展，对应不支持断点相关功能与组件（TM） 当配置为 ON 时，支持 SDTRIG 扩展，支持断点相关功能与组件 <i>注：这个参数配置为 ON 时，仅当 SDEXT_EXT 也配置为 ON 时生效</i>
NTRACE_EXT	架构类	ON/OFF	OFF	当配置为 OFF 时，不支持 NTrace 相关功能与模块 当配置为 ON 时，支持 NTrace 相关功能与模块
NMI_EXT	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 NMI 中断输入与相关处理 当配置为 ON 时，支持 NMI 中断输入与相关处理 <i>注：NMI 没有特殊声明的情况下，默认是不可恢复的 NMI</i>
RNMI_EXT	架构类	ON/OFF	OFF	当配置为 OFF 时，不支持可恢复 NMI

				当配置为 ON 时，支持可恢复 NMI <i>注：这个参数配置为 ON 时，仅当 NMI_EXT 也配置为 ON 时生效</i>
架构类参数				
U_MOD	架构类	ON/OFF	ON	当配置为 OFF 时，特权模式不支持 U Mode，支持默认的 M Mode 当配置为 ON 时，特权模式支持 U Mode 和 M Mode
PMP	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 PMP 相关功能 当配置为 ON 时，支持 PMP 相关功能 <i>注：这个参数配置为 ON 时，仅当 U_MOD 也配置为 ON 时生效</i>
PMA	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 PMA 相关功能，所有空间使用默认 memory 属性 当配置为 ON 时，支持 PMA 相关功能
PMP_NAPOT_ONLY	架构类	ON/OFF	ON	当配置为 OFF 时，PMP 同时支持 TOR 和 NAPOT 两种地址 match 模式 当配置为 ON 时，PMP 仅支持 NAPOT 地址 match 模式
ICACHE	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 ICache 相关功能 当配置为 ON 时，支持 ICache 相关功能
ITCM	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 ITCM 相关功能 当配置为 ON 时，支持 ITCM 相关功能
DTCM	架构类	ON/OFF	ON	当配置为 OFF 时，不支持 DTCM 相关功能 当配置为 ON 时，支持 DTCM 相关功能
ECC	架构类	ON/OFF	ON	当配置为 OFF 时，不支持针对 Memory 的 ECC 校验 当配置为 ON 时，支持针对 Memory 的 ECC 校验
ISSUE_1	架构类	ON/OFF	ON	当配置为 ON 时，后端流水线实现为单发 <i>注：这个宏与 ISSUE_2 互斥，且必须定义一个</i>
ISSUE_2	架构类	ON/OFF	OFF	当配置为 ON 时，后端流水线实现为双发 <i>注：这个宏与 ISSUE_1 互斥，且必须定义一个</i>
FAST_MUL	架构类	ON/OFF	ON	当配置为 OFF 时，乘法指令使用多周期迭代加法实现 当配置为 ON 时，乘法指令使用单周期 booth

				乘法实现 <i>注意：这个参数仅在 M_EXT 为 ON 时生效</i>
DM_HAS_PROG	架构类	ON/OFF	ON	当配置为 OFF 时，调试模块不支持 Program Buffer 及相关特性 当配置为 ON 时，调试模块支持 Program Buffer 及相关特性 <i>注意：这个参数仅在 SDEXT_EXT 为 ON 时生效</i>
EMBEDDED_TIMER	架构类	ON/OFF	ON	当配置为 OFF 时，不支持内嵌的 timer，time 信息由顶层接口输入 当配置为 ON 时，支持内嵌的 timer（基于输入的参考时钟），不需要 SoC 输入计时信息
接口类参数				
EXTERNAL_MEM_WRAP	接口类	ON/OFF	ON	当配置为 OFF 时，Memory Wrap 内嵌在 M130 顶层内（无 Memory 相关顶层接口） 当配置为 ON 时，Memory Wrap 外挂在 M130 顶层外（存在 Memory 相关顶层接口） <i>注意：这个参数仅在 ICACHE/ITCM/DTCM 等 Memory 相关宏配置为 ON 时有效</i>
I_BUS	接口类	ON/OFF	ON	当配置为 OFF 时，不支持对外的 I_BUS 接口（AHB_Lite） 当配置为 ON 时，支持对外的 I_BUS 接口（AHB_Lite）
D_BUS	接口类	ON/OFF	ON	当配置为 OFF 时，不支持对外的 D_BUS 接口（AHB_Lite） 当配置为 ON 时，支持对外的 D_BUS 接口（AHB_Lite）
M_BUS	接口类	ON/OFF	ON	当配置为 OFF 时，不支持对外的 M_BUS 接口（AHB_Lite） 当配置为 ON 时，支持对外的 M_BUS 接口（AHB_Lite）
P_BUS	接口类	ON/OFF	ON	当配置为 OFF 时，不支持对外的 P_BUS 接口（APB） 当配置为 ON 时，支持对外的 P_BUS 接口（APB）
S_BUS	接口类	ON/OFF	ON	当配置为 OFF 时，不支持对外的 S_BUS 接口

				(AHB_Lite) 当配置为 ON 时, 支持对外的 S_BUS 接口 (AHB_Lite)
CJTAG	接口类	ON/OFF	OFF	当配置为 OFF 时, 支持 4/5 线 JTAG 调试接口 当配置为 ON 时, 支持 2 线 JTAG 调试接口
数值类参数				
PMP_ENTRY_N	数值类	1~16	8	PMP entry 的数量配置 注意: 这个参数仅 PMP 配置为 ON 时生效
PMA_ENTRY_N	数值类	1~16	7	PMA entry 的数量配置 注意: 这个参数仅 PMA 配置为 ON 时生效
ICACHE_SIZE	数值类	(1~32)*1024	8*1024	ICACHE 的容量配置, 步长单位为 1024B 注意: 这个参数仅 ICACHE 配置为 ON 时生效
ITCM_SIZE	数值类	(1~256)*1024	4*1024	ITCM 的容量配置, 步长单位为 1024B 注意: 这个参数仅 ITCM 配置为 ON 时生效
DTCM_SIZE	数值类	(1~256)*1024	4*1024	DTCM 的容量配置, 步长单位为 1024B 注意: 这个参数仅 DTCM 配置为 ON 时生效
EXT_INT_N	数值类	1~128	32	外部中断数量个数 注意 1: 这个参数仅当 SMCLIC_EXT 配置为 ON 时生效 注意 2: 这里的中断个数配置仅包含外部中断, 不包含核内相关中断
TRIG_N	数值类	1~16	4	Trigger 个数 注意: 这个参数仅当 SDTRIG_EXT 配置为 ON 时生效
ZICNTR_W	数值类	1~64	24	定义 Timer 以及内核中 cycle/timer/instret CSR 的位宽

2.2. 典型配置

Wing-M130 基于不同的性能与 PPA 目标, 支持默认四个档位配置, 对应 M130/M132/M134/M136 四个子版本, 其中数字越大的版本, 性能越好, 同时面积功耗代价也越大。四个子版本的主要配置差异如下:

➤ M130

支持 RV32EM_ZC_XC_B_ZICOND_ZICSR 基础指令集

支持单发射

支持快速乘法实现

支持 U mode 与 PMP/PMA

不支持 TCM/Cache

支持 CLIC 中断控制器，支持 16 个外部中断与 1 个 NMI 中断

支持 DM/TM 等调试组件与 JTAG 调试接口，不支持 Trace

支持 32bit 内嵌 Timer

支持基于 WFI 的低功耗模式

➤ M132

支持 RV32IM_ZC_XC_B_ZICOND_ZICSR 基础指令集

支持单发射

支持快速乘法实现

支持 U mode 与 PMP/PMA

不支持 TCM/Cache

支持 CLIC 中断控制器，支持 16 个外部中断与 1 个 NMI 中断

支持 DM/TM 等调试组件与 JTAG 调试接口，不支持 Trace

支持 32bit 内嵌 Timer

支持基于 WFI 的低功耗模式

➤ M134

支持 RV32IM_ZC_XC_B_ZICOND_ZICSR 基础指令集

支持双发射

支持快速乘法实现

支持 U mode 与 PMP/PMA

支持 ITCM/DTCM，默认各 32KB

支持 ITCM/DTCM ECC 保护

不支持 ICache

支持 CLIC 中断控制器，支持 16 个外部中断与 1 个 NMI 中断

支持 DM/TM 等调试组件与 JTAG 调试接口，不支持 Trace

支持 32bit 内嵌 Timer

支持基于 WFI 的低功耗模式

➤ M136

支持 RV32IM_ZC_XC_B_ZICOND_ZICSR 基础指令集

支持 A 扩展（排他与原子操作指令）

支持 Zfinx 扩展（支持单精度浮点）

支持双发射

支持快速乘法实现

支持 U mode 与 PMP/PMA

支持 ITCM/DTCM，默认各 32KB

支持 ICache，默认 16KB

支持 ITCM/DTCM/ICache ECC 保护

支持 CLIC 中断控制器，支持 16 个外部中断与 1 个 NMI 中断

支持 DM/TM 等调试组件与 JTAG 调试接口，支持 Trace

支持 32bit 内嵌 Timer

支持基于 WFI 的低功耗模式

四个子版本的性能与面积数据参考如下：

Table 2-2 Wing-M130 各版本跑分&面积数据参考

	M130	M132	M134	M136
Dhrystone 跑分 (单位 DMIPS)	1.47	1.63	2.21	2.21
Coremark 跑分 (单位 Coremark/MHz)	3.02	3.13	4.08	4.13
面积 (单位 μm^2)	0.029	0.031	0.046	0.063

备注 1: Dhrystone 采取标准-O2 -fno-inline 编译选项

备注 2: 面积为 T28 200Mhz ssg0p81vm40c corner 下评估得到，仅供参考

备注 3: 面积只包含逻辑部分，

3. 接口信号描述

3.1. 时钟与复位信号

Wing-M130 时钟复位相关信号列表如下：

Table 3-1 时钟复位信号列表

Signal	Direction	Width	Description
Clock signals			
always_on_clk	input	1	系统常开时钟
clk	input	1	M130 主时钟 M130 内核以及子系统（除 JTAG/DTM/Timer 等逻辑外）都运行在这个主时钟域下
ref_clk	input	1	SoC 输入的慢速参考时钟，用于内嵌 Timer 的计时 <i>注意：这个接口仅当 EMBEDDED_TIMER 参数配置为 ON 时存在</i>
clk_gate_en	output	1	M130 指示 SoC 可以门控输入的 clk 时钟
Reset signals			
pwrup_rst_n	input	1	M130 上电复位信号，异步复位，低有效 控制 M130 内部所有时序逻辑的复位
ndm_rst_n	input	1	M130 Non-DM 复位信号，同步复位，低有效 控制 M130 内部除 DM/DTM/DMI(JTAG)调试外的逻辑复位
dm_ndmrst	output	1	调试模块向 SoC 发出 Non-DM 复位请求信号，SoC 需要基于这个请求，在一定周期内控制产生 ndm_rst_n 复位信号
soft_rst_n	input	1	M130 软复位信号，同步复位，低有效 控制 M130 内部除 DM/DTM/DMI(JTAG)/中断控制器/TM 外的逻辑复位 备注：受软复位特性宏控制，当没有配置支持软复位宏时，不支持此接口
soft_rst_req	output	1	M130 向 SoC 发出软复位请求信号，SoC 需要基于这个请求，在一定周期内控制产生 soft_rst_n 复位信号 备注：受软复位特性宏控制，当没有配置支持软复位宏时，不支持此接口

3.2. 调试接口信号

Wing-M130 默认支持标准 5 线 JTAG 调试接口（当未配置 CJTAG 宏时），用于访问调试模块内的寄存器。5 线 JTAG 符合 IEEE 1149.1 协议，信号列表如下：

Table 3-2 五线 JTAG 调试接口列表

Signal	Direction	Width	Description
trst_n	input	1	JTAG 接口复位信号，低有效
tck	input	1	JTAG 接口时钟信号
tms	input	1	JTAG 接口 Test Mode Select 信号
tdi	input	1	JTAG 接口 Test Data input 信号
tdo	output	1	JTAG 接口 Test Data output 信号
tdo_en	output	1	JTAG 接口 Test Data output Enable 信号，高有效

当配置 CJTAG 宏时，Wing-M130 改为支持两线 CJTAG，用于访问调试模块内的寄存器。两线 CJTAG 符合 IEEE 1149.7 协议，信号列表如下：

Table 3-3 两线 CJTAG 调试接口列表

Signal	Direction	Width	Description
trst_n	input	1	JTAG 接口复位信号，低有效
tck	input	1	JTAG 接口时钟信号
tms	input	1	JTAG 接口 Test Mode Select 信号
tdo	output	1	JTAG 接口 Test Data output 信号
tdo_en	output	1	JTAG 接口 Test Data output Enable 信号，高有效

注：CJTAG 两线接口中的 tms 为 inout 类型，Wing-M130 配置为 CJTAG 调试接口时，不在内部实现三态门，因此定义了 tms+tdo+tdo_en 三个信号，SoC 需要实现对应的三态门（并支持 keeper），将这三个信号转换为一个 inout 信号。

3.3. 中断接口信号

M130 中断主要包括外部中断线输入，不可屏蔽中断线输入，以及中断配置信号等，具体如下：

Table 3-4 中断接口列表

Signal	Direction	Width	Description
loc_interrupt	input	EXT_INT_N (默认值 32)	SoC 输入的外部 Local 中断线 此信号位宽由 EXT_INT_N 配置（即外部中断数量）决

			定 注：仅当 <i>SMCLIC_EXT</i> 宏配置有效时，这个信号存在
ext_interrupt	input	1	SoC 输入的外部中断线，对应上报到内核中的 MEIP
nmi	input	1	不可屏蔽中断线 注：仅当 <i>NMI</i> 宏配置有效时，这个信号存在
mnvec	input	32	不可屏蔽中断的 Trap 地址 注1：仅当 <i>NMI</i> 宏配置有效时，这个信号存在 注2：此信号要求内核解复位前稳定，且保持不变
rnmi_id	input	32	RNMI 的中断 ID 号 注：仅当 <i>RNMI</i> 宏配置有效时，这个信号存在

3.4. 外部数据接口信号(Master)

Wing-M130 支持 3 组位宽 32bit 的对外 AHB_Lite 数据总线接口，用于内核取指/访存/调试请求访问 SoC 中的存储，分别为：

- I_BUS：用于取指请求访问指令空间
- D_BUS：用于访存/调试请求访问指令空间
- M_BUS：用于访存/调试请求访问 SoC 其他空间

Wing-M130 还额外支持一个 32bit APB 接口 (P_BUS)，用于内核访问 SoC 的外设空间

注1：上述为 *Wing-M130* 推荐的对外总线接口使用方式，用户可以根据实际需要，灵活配置对外接口形态，具体参考后文描述。

注2：上述接口，受到对应宏开关控制，当对应宏没有定义时，对应的总线及其接口信号都不存在

具体的接口信号列表如下：

Table 3-5 外部数据接口信号列表

Signal	Direction	Width	Description
I-BUS (AHB-Lite)			
i_htrans	output	2	指示当前 AHB 总线传输的类型，编码如下： 00 – IDLE 01 – BUSY 10 – NONSEQ 11 – SEQ
i_haddr	output	32	指示当前 AHB 总线传输的地址
i_hwrite	output	1	指示当前 AHB 总线传输的读写类型

			0 – 读请求 1 – 写请求 <i>注 1：在默认配置下，I_BUS 不支持写传输，这个信号输出固接 0</i>
i_hburst	output	3	指示当前 AHB 总线的 Burst 传输类型 Wing-M130 只支持 single 传输，不支持 burst，因此这个信号固接 3'b000
i_hsize	output	3	指示当前 AHB 总线的传输 size 000 – Byte 001 – Halfword 010 – Word
i_hprot	output	4	指示当前 AHB 总线的传输 Memory 属性和 Protection 属性，编码如下： bit0 - 指令/数据访问，0 表示指令访问，1 表示数据访问 bit1 - 特权模式，0 表示 user mode，1 表示 machine mode bit2 - Bufferable 属性，0 表示 non-bufferable，1 表示 bufferable bit3 - Cacheable 属性，0 表示 non-cacheable，1 表示 cacheable
i_hwdata	output	32	指示当前 AHB 总线的传输写数据 <i>注 1：在默认配置下，I_BUS 不支持写传输，这个信号输出固接 0</i>
i_hready	input	1	指示当前 AHB 总线传输握手/完成信号
i_hrdata	input	32	指示当前 AHB 总线传输的读数据信号
i_hresp	input	1	指示当前 AHB 总线传输的响应信息，编码如下： 0 - OKAY 响应 1 - ERROR 响应
i_hmaster	output	2	当前 I-Bus 上传输请求的来源： 0 - 普通 load/store 指令触发 1 - Debug 请求触发 2 - Instruction Fetch 触发 3 - 访问向量中断入口触发
D-BUS (AHB-Lite)			
d_htrans	output	2	指示当前 AHB 总线传输的类型，编码如下： 00 – IDLE 01 – BUSY 10 – NONSEQ

			11 – SEQ
d_haddr	output	32	指示当前 AHB 总线传输的地址
d_hwrite	output	1	指示当前 AHB 总线传输的读写类型 0 – 读请求 1 – 写请求
d_hburst	output	3	指示当前 AHB 总线的 Burst 传输类型 Wing-M130 只支持 single 传输，不支持 burst，因此这个信号固接 3'b000
d_hsize	output	3	指示当前 AHB 总线的传输 size 000 – Byte 001 – Halfword 010 – Word
d_hprot	output	4	指示当前 AHB 总线的传输 Memory 属性和 Protection 属性，编码如下： bit0 - 指令/数据访问，0 表示指令访问，1 表示数据访问 bit1 - 特权模式，0 表示 user mode，1 表示 machine mode bit2 - Bufferable 属性，0 表示 non-bufferable，1 表示 bufferable bit3 - Cacheable 属性，0 表示 non-cacheable，1 表示 cacheable
d_hwdata	output	32	指示当前 AHB 总线的传输写数据
d_hready	input	1	指示当前 AHB 总线传输握手/完成信号
d_hrdata	input	32	指示当前 AHB 总线传输的读数据信号
d_hresp	input	1	指示当前 AHB 总线传输的响应信息，编码如下： 0 – OKAY 响应 1 – ERROR 响应
d_hmaster	output	2	当前 D-Bus 上传输请求的来源： 0 - 普通 load/store 指令触发 1 - Debug 请求触发 2 - Instruction Fetch 触发 3 – 访问向量中断入口触发
M-BUS (AHB-Lite)			
m_htrans	output	2	指示当前 AHB 总线传输的类型，编码如下： 00 – IDLE 01 – BUSY 10 – NONSEQ 11 – SEQ

m_haddr	output	32	指示当前 AHB 总线传输的地址
m_hwrite	output	1	指示当前 AHB 总线传输的读写类型 0 – 读请求 1 – 写请求
m_hburst	output	3	指示当前 AHB 总线的 Burst 传输类型 Wing-M130 只支持 single 传输, 不支持 burst, 因此这个信号固接 3'b000
m_hsize	output	3	指示当前 AHB 总线的传输 size 000 – Byte 001 – Halfword 010 – Word
m_hprot	output	4	指示当前 AHB 总线的传输 Memory 属性和 Protection 属性, 编码如下: bit0 - 指令/数据访问, 0 表示指令访问, 1 表示数据访问 bit1 - 特权模式, 0 表示 user mode, 1 表示 machine mode bit2 - Bufferable 属性, 0 表示 non-bufferable, 1 表示 bufferable bit3 - Cacheable 属性, 0 表示 non-cacheable, 1 表示 cacheable
m_hwdata	output	32	指示当前 AHB 总线的传输写数据
m_hready	input	1	指示当前 AHB 总线传输握手/完成信号
m_hrdata	input	32	指示当前 AHB 总线传输的读数据信号
m_hresp	input	1	指示当前 AHB 总线传输的响应信息, 编码如下: 0 – OKAY 响应 1 – ERROR 响应
m_hmstlock	output	1	当前 M-Bus 上传输的操作需要保持原子性
m_hmaster	output	2	当前 M-Bus 上传输请求的来源: 0 - 普通 load/store 指令触发 1 - Debug 请求触发 2 - Instruction Fetch 触发 3 – 访问向量中断入口触发
P-BUS (APB)			
p_penable	output	1	P_BUS 传输使能信号
p_psel	output	1	P_BUS 传输选通信号
p_pready	input	1	P_BUS 传输被接受信号
p_paddr	output	32	P_BUS 传输访问地址
p_pwrite	output	1	P_BUS 传输读写指示信号, 编码如下

			0: 读 1: 写
p_pwdata	output	32	P_BUS 传输写数据
p_pwstrb	output	4	P_BUS 传输写 strb
p_prdata	input	32	P_BUS 传输读数据
p_pslverr	input	1	P_BUS 传输响应 0: 正常响应 1: 异常响应

3.5. 外部数据接口信号(Slave)

Wing-M130 支持 1 组位宽 32bit 的 AHB Slave 接口，用于 SoC 访问内部寄存器/TCM Memory

注：受到 S-BUS 开关控制，当对应宏没有定义时，对应的总线及其接口信号都不存在

接口列表如下：

Table 3-6 外部 Slave 接口信号

Signal	Direction	Width	Description
S-BUS (AHB-Lite)			
s_htrans	input	2	指示当前 AHB 总线传输的类型，编码如下： 00 – IDLE 01 – BUSY 10 – NONSEQ 11 – SEQ
s_haddr	input	32	指示当前 AHB 总线传输的地址
s_hwwrite	input	1	指示当前 AHB 总线传输的读写类型 0 – 读请求 1 – 写请求
s_hburst	input	3	指示当前 AHB 总线的 Burst 传输类型 Wing-M130 只支持 single 传输，不支持 burst，因此这个信号固接 3'b000
s_hsize	input	3	指示当前 AHB 总线的传输 size 000 – Byte 001 – Halfword 010 – Word
s_hprot	input	4	指示当前 AHB 总线的传输 Memory 属性和 Protection 属性，编码如下：

			bit0 - 指令/数据访问, 0 表示指令访问, 1 表示数据访问 bit1 - 特权模式, 0 表示 user mode, 1 表示 machine mode bit2 - Bufferable 属性, 0 表示 non-bufferable, 1 表示 bufferable bit3 - Cacheable 属性, 0 表示 non-cacheable, 1 表示 cacheable
s_hwdata	input	32	指示当前 AHB 总线的传输写数据
s_hready	output	1	指示当前 AHB 总线传输握手/完成信号
s_hrdata	output	32	指示当前 AHB 总线传输的读数据信号
s_hresp	output	1	指示当前 AHB 总线传输的响应信息, 编码如下: 0 – OKAY 响应 1 – ERROR 响应

3.6. 其他接口信号

Wing-M130 支持其他信号列表如下:

Table 3-7 其他接口信号列表

Signal	Direction	Width	Description
Misc Interface			
boot_pc	input	32	Wing-M130 核的启动 PC, 决定复位后第一执行指令的 PC。 注1: 此信号要求 2B 对齐 注2: 此信号要求内核解复位前稳定, 且保持不变
core_itcm_base_addr	input	32	M130 ITCM MMR 空间基地址 注1: 基地址要求 256KB 对齐 注2: 此信号要求内核解复位前稳定, 且保持不变 注3: 此信号仅在配置了 ITCM 的情况下存在
core_dtcmm_base_addr	input	32	M130 DTCM MMR 空间基地址 注1: 基地址要求 256KB 对齐 注2: 此信号要求内核解复位前稳定, 且保持不变 注3: 此信号仅在配置了 ITCM 的情况下存在
core_mmr_base_addr	input	32	M130 Memory Mapped Register 空间基地址

			注1: 基地址要求 256KB 对齐 注2: 此信号要求内核解复位前稳定, 且保持不变
hard_id	input	32	SoC 输入的核号, 会上报到内核 mhartid CSR 上 注: 此信号要求内核解复位前稳定, 且保持不变
rer_bank_inst_id	input	16	SoC 输入的 RERI Bank ID, 会上报到 RERI (RAS 错误记录模块) 的寄存器中 注: 此信号要求内核解复位前稳定, 且保持不变
core_wait	input	1	SoC 输入, 拉高时, 指示内核在解复位后, 暂停取指行为, 直到这个信号拉低, 内核再启动取指与程序执行。 注1: 这个信号在拉高期间, 内核停止正常程序执行, 但是可以响应/进入 debug 模式并进行调试 注2: 这个信号在核开始执行后不允许拉高
endianess	input	1	SoC 输入, 控制内核部分操作的大小端 1: 大端 0: 小端 注意: 要求 SoC 保证这个信号为静态稳定信号后, 再解复位内核, 并且再内核运行过程中保持稳定不变
halted	output	1	指示内核处于调试模式, 高电平有效
lockup	output	1	指示内核处于不可恢复异常状态中, 高电平有效
timer_calibration	input	34	Timer 校准信息 bit[31:0] 参考时钟计时 10ms 所需的周期数 bit[32]: Inexact, 0 表示当前 timer 可以精确计时 10ms, 否则为 1 bit[33]: NOREF, 表示没有可选时钟源
dbg_authen	input	1bit	Debug 鉴权通过指示信号 当此信号输入为 1 时, Wing-M130 调试相关功能可以正常使用 当此信号输入为 0 时, Wing-M130 调试大部分功能被禁止, 仅可以通过调试接口读取调试相关寄存器状态 注: 此信号要求解复位前稳定

3.7. 低功耗接口

Wing-M130 支持配置一组基于 AMBA Low Power 协议定义的 Q-Channel 接口，与 SoC 中的功耗管理单元交互，控制 Wing-M130 进入/推出低功耗模式。当没有配置 Q-Channel 时，Wing-M130 仅支持 sleeping 和 deep_sleeping 状态上报，具体低功耗（时钟关断）设计由 SoC 完成

具体信号定义如下：

Table 3-8 低功耗信号列表

Signal	Direction	Width	Description
Low Power Interface			
core_qreq_n	input	1	SoC 输入的低功耗握手请求信号 当拉低时，表示 SoC 请求 M130 进入低功耗状态。 当拉高时，表示 SoC 请求 M130 退出低功耗状态。 具体参考 AMBA Low Power Interface Q-Channel 定义 注：仅当配置支持 Q-Channel 时存在此接口
core_qaccept_n	output	1	当 SoC 输入的 core_qreq_n 拉低，请求 M130 进入低功耗状态后，当 M130 满足条件可以进入低功耗状态后，拉低此信号，否则保持不变。 当 SoC 输入的 core_qreq_n 拉高，请求 M130 退出低功耗状态后，当 M130 满足条件可以退出低功耗状态后，拉高此信号，否则保持不变。 具体参考 AMBA Low Power Interface Q-Channel 定义 注：仅当配置支持 Q-Channel 时存在此接口
core_qdeny	output	1	当 SoC 输入的 core_qreq_n 拉低，请求 M130 进入低功耗状态后，当 M130 不满足条件，无法进入低功耗状态后，拉高此信号，否则保持不变。 具体参考 AMBA Low Power Interface Q-Channel 定义 注：仅当配置支持 Q-Channel 时存在此接口
core_qactive	output	1	M130 内核处于 active 状态指示 当 M130 内核完成解复位开始运行后，再未进入 WFI 状态期间，都会持续拉高此状态。 当 M130 内核进入 WFI 低功耗状态后，将拉低此信号，表示内核已进入低功耗模式

			注：仅当配置支持 Q-Channel 时存在此接口
core_sleeping	output	1	指示 M130 内核处于 Sleeping 状态，SoC 可以关闭 M130 时钟
core_deep_sleeping	output	1	指示 M130 内核处于 Deep Sleeping 状态，SoC 可以关闭 M130 时钟

3.8. Memory Wrap 接口信号

当配置 EXTERNAL_MEM_WRAP 宏时，Wing-M130 的 Memory Wrap 采取外挂的方式实现，顶层支持如下接口信号：

注：仅当配置 ITCM/DTCM/ICache 相关宏时，也即 Wing-M130 存在 Memory 时，支持对应的 Memory Wrapper 接口

注：Memory 的位宽/深度 受多参数影响，下表仅为默认配置下的位宽值

Table 3-9 Memory Wrap 信号列表

Signal	Direction	Width	Description
ITCM Memory Wrapper signals			
itcm_sram_cs	output	2	ITCM Memory 片选信号，高有效
itcm_sram_wr	output	2	ITCM Memory 访问操作类型 1 – 写 0 – 读
itcm_sram_addr	output	2*9	ITCM Memory 访问地址 注：地址位宽受 ITCM 实际容量影响
itcm_sram_wdata	output	2*32	ITCM Memory 的写数据
itcm_sram_wen	output	2*32	ITCM Memory 的写 bit enable, 每个 bit 对应一个 data bit 的写使能
itcm_sram_rdata	input	2*32	ITCM Memory 的读数据
DTCM Memory Wrapper signals			
dtdcm_sram_cs	output	1	DTCM Memory 片选信号，高有效
dtdcm_sram_wr	output	1	DTCM Memory 访问操作类型 1 – 写 0 – 读
dtdcm_sram_addr	output	10	DTCM Memory 访问地址 注：地址位宽受 DTCM 实际容量影响
dtdcm_sram_wdata	output	32	DTCM Memory 的写数据
dtdcm_sram_wen	output	32	DTCM Memory 的写 bit enable, 每个 bit 对应一个 data bit 的写使能

dtcm_sram_rdata	input	32	DTCM Memory 的读数据
ICACHE Memory Wrapper signals			
ic_sram_tag_cs	output	1	ICACHE TAG CS Memory 片选信号, 高有效
ic_sram_tag_wr	output	1	ICACHE TAG CS Memory 访问操作类型 1 – 写 0 – 读
ic_sram_tag_addr	output	9	ICACHE TAG CS Memory 访问地址 注: 地址位宽受 ICache 实际容量影响
ic_sram_tag_wdata	output	20	ICACHE TAG CS Memory 的写数据
ic_sram_tag_wen	output	20	ICACHE TAG CS Memory 的写 bit enable, 每个 bit 对应一个 data bit 的写使能
ic_sram_tag_rdata	input	20	ICACHE TAG CS Memory 的读数据
ic_sram_data_cs	output	1	ICACHE DATA CS Memory 片选信号, 高有效
ic_sram_data_wr	output	1	ICACHE DATA CS Memory 访问操作类型 1 – 写 0 – 读
ic_sram_data_addr	output	9	ICACHE DATA CS Memory 访问地址 注: 地址位宽受 ICache 实际容量影响
ic_sram_data_wdata	output	64	ICACHE DATA CS Memory 的写数据
ic_sram_data_wen	output	64	ICACHE DATA CS Memory 的写 bit enable, 每个 bit 对应一个 data bit 的写使能
ic_sram_data_rdata	input	64	ICACHE DATA CS Memory 的读数据
Trace Memory Wrapper signals			
itrace_sram_cs	output	2*1	ITRACE Memory 片选信号, 高有效
itrace_sram_wr	output	2*1	ITRACE Memory 访问操作类型 1 – 写 0 – 读
itrace_sram_addr	output	2*10	ITRACE Memory 访问地址 注: 地址位宽受 ITRACE MEMORY 实际容量影响
itrace_sram_wdata	output	2*32	ITRACE Memory 的写数据
itrace_sram_wen	output	2*32	ITRACE Memory 的写 bit enable, 每个 bit 对应一个 data bit 的写使能
itrace_sram_rdata	input	2*32	ITRACE Memory 的读数据

3.9.DFT 接口信号

DFT 相关信号列表如下:

Table 3-10 DFT 信号列表

Signal	Direction	Width	Description
DFT signals			
dft_scan_mode	input	1	DFT 扫描模式选择信号
dft_icg_scan_en	input	1	DFT 扫描使能信号
dft_scan_rst_n	input	1	DFT 扫描复位信号

4. 关键特性集成说明

4.1. 时钟与复位

4.1.1. 时钟系统集成

Wing-M130 支持两个主时钟输入信号，这两个输入时钟要求同源

- `always_on_clk`，内核常开时钟，用于部分在低功耗模式下时钟不能关断的逻辑（例如中断采样逻辑，Debug 逻辑等）
- `clk`，内核中除 DMI(JTAG 调试接口相关逻辑) 以及内嵌 Timer (如果配置支持)，以及上述 `always_on_clk` 时钟域内的逻辑外，所有逻辑均工作在这个主时钟下。

Wing-M130 支持输出一个时钟使能控制信号 `clk_gate_en`，当内核处于 WFI 低功耗状态下时，且 `mcg_disable` 没有被置位时，会拉高 `clk_gate_en` 信号，指示外部 SoC 可以基于此信号对主时钟 (`clk`) 进行门控。

当 Wing-M130 通过 `EMBEDDED_TIMER` 宏配置了内嵌 Timer 功能后，需要 SoC 输入一个额外的慢速参考时钟信号，`ref_clk`，用于 Timer 计数。`ref_clk` 时钟与核的主时钟不同源，Wing-M130 会对齐进行异步处理后再实现 Timer 相关功能。由于 RISC-V 对于 Timer 的定义，需要 SoC 保证参考时钟的稳定性，不支持动态调频等特性，保证 Timer 计时的精准度。

Wing-M130 要求 `ref_clk` 时钟频率至少低于内核主时钟的 1/5。

Wing-M130 还额外支持一个 `tck` 信号，由调试接口输入，用于 DMI(JTAG)调试部分逻辑的时钟。

当前版本约束 JTAG 的 `tck` 时钟主频，不能高于内核主时钟 `clk` 的 1/4，cJTAG 的 `tck` 时钟主频，不能高于内核主时钟 `clk` 的 1/3

4.1.2. 复位系统集成

Wing-M130 支持四个输入复位：

- `pwrup_rst_n`：整个 Wing-M130 的上电复位输入，异步复位，M130 内部会对其进行同步撤离处理，低有效。
- `ndm_rst_n`：复位除 DM/DTM/DMI(JTAG)外的逻辑，同步复位，低有效。通过调试接口配置 `ndm reset` 后，M130 会通过输出信号 `dm_ndmrst` 指示 SoC，SoC 在采样到此信号为高电平后，在若干 cycle 内需要通过拉低 `ndm_rst_n` 信号对 M130 进行复位。
- `soft_rst_n`：复位除 DM/DTM/DMI(JTAG)/中断控制器/TM 外的逻辑，同步复位，低有效。通过写 M130 内部 MMR 寄存器后，M130 会通过输出信号 `soft_rst_req` 指示 SoC，SoC 在采样到此信号为高电平后，在若干 cycle 内需要通过拉低 `soft_rst_n` 信号对 M130 进行软复位。
- `trst_n`：JTAG 调试接口复位信号，低有效。

Wing-M130 的复位系统框图如下所示：

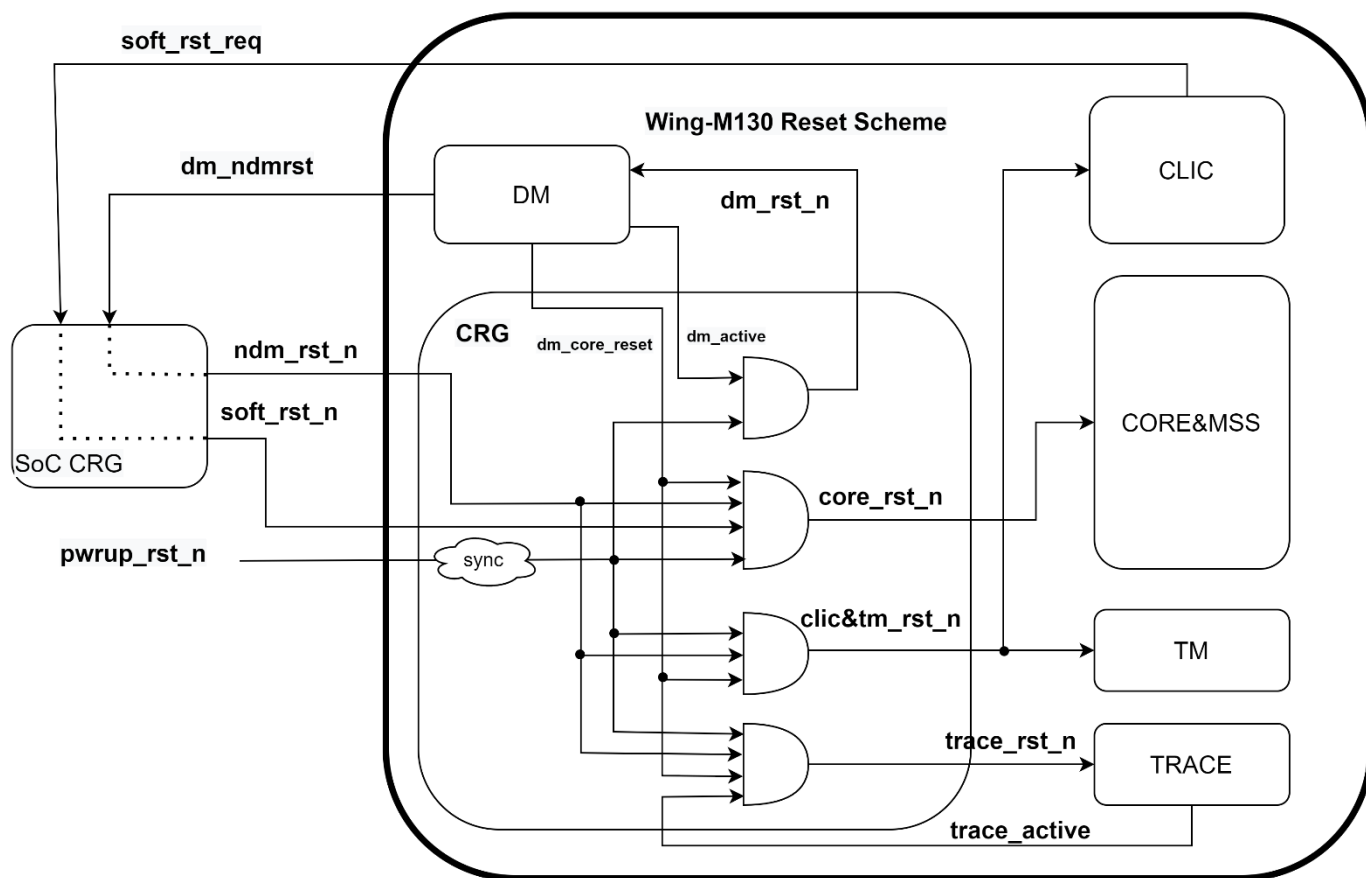


Figure 4-1 Wing-M130 复位系统示意图

4.1.2.1. 上电复位

当系统启动时，需要首先拉低上电复位信号 `pwrup_rst_n`，并在内核的主时钟打开后，至少五个 cycle 后解除上电复位，Wing-M130 提供了针对 `pwrup_rst_n` 的同步撤离设计，因此外部系统可以将 `pwrup_rst_n` 复位实现为异步复位。

4.1.2.2. NDM 复位

Wing-M130 支持由调试系统发起的 NDM 复位，调试软件通过 JTAG 接口控制调试模块发出 `dm_ndmrst` 信号给到 SoC，此信号为电平信号，高电平时表示请求复位整个除调试系统外的逻辑，低电平表示解除复位。SoC 集成时可以根据实际需要定义这个信号产生的复位控制范围，但是需要至少通过 `ndm_rst_n` 信号给到 Wing_M130 复位指示。

注意：SoC 系统需要保证从 `dm_ndmrst` 信号拉高后，一定延迟内，拉低 `ndm_rst_n` 信号，复位 Wing-M130 (Debug 逻辑除外)，在 `dm_ndmrst` 信号拉低后，一定延迟内，拉高 `ndm_rst_n` 信号，解除 Wing-M130 复位。此延迟由 SoC 实现决定，但建议不超过 10 个核时钟周期

Dm_ndmrst 信号为与内核时钟同步的信号，Wing-M130 不会针对输入的 ndm_rst_n 进行同步撤离处理，需要 SoC 保证。

4.1.2.3. 软复位

Wing-M130 支持由通过写 MMR 寄存器发起软复位，软复位寄存器 srcr 地址为 MCLINTBASE+0x30。软件写此寄存器的最低 bit 值 1，可以向 SoC 发起软复位请求。

注意：SoC 系统需要保证从信号拉高后，一定延迟内，拉低 soft_rst_n 信号，复位 Wing-M130（调试与中断逻辑除外）。此延迟由 SoC 实现决定，但建议不超过 10 个核时钟周期。当内核收到软复位信号后，会一通复位掉 srcr 寄存器，从而自动拉低软复位请求信号。

Soft_rst_n 信号为与内核时钟同步的信号，Wing-M130 不会针对输入的 soft_rst_n 进行同步撤离处理，需要 SoC 保证。

4.1.2.4. 内部复位

Wing-M130 内部存在如下复位机制：

- 调试器通过写 DM 中 DmControl.hartreset 域触发，内部自动产生一个同步复位信号，并复位 Wing-M130 内部除 DM 外的所有逻辑
- 调试器通过写 DM 中 DmControl.dmactive 域触发，内部自动产生一个同步复位信号，并复位 Wing-M130 中 DM 相关逻辑（不包含 DmControl.dmactive 本身）
- 调试器/软件通过写 Trace 中 TeActive 域触发，内部自动产生一个同步复位信号，并复位 Wing-M130 中 Trace 相关逻辑（不包含 TeActive 本身）

4.1.3. 内核 Boot 流程

4.1.3.1. Boot From SoC 流程

如果用户希望从 SoC 空间启动 Wing-M130，推荐 Boot 流程如下：

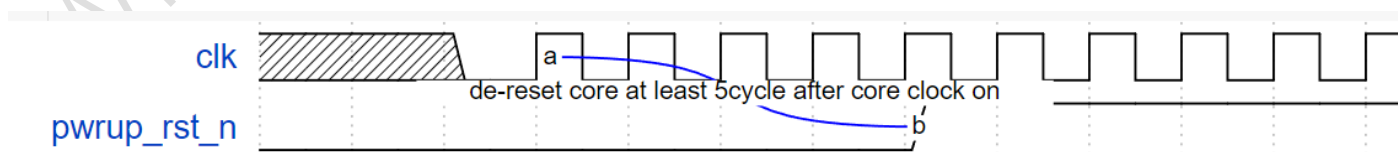


Figure 4-2 Wing-M130 Boot From SoC 时序图

需要注意：如果配置了 ICache，Wing-M130 有一段硬件初始化的过程，当 SoC 解复位之后，内核需要经过若干个 cycle，才会开始执行

4.1.3.2. Boot From TCM 流程

如果用户希望从 TCM 空间（如果配置了 TCM）启动 Wing-M130，推荐如下方式。

Boot 流程 2：用户可以通过拉高 `core_wait` 信号，在解复位后让核保持 `stall` 状态，直到通过 S-Bus 完成 TCM 初始化后，拉低 `core_wait` 信号，内核开始从 Boot PC 开始执行。具体如下图所示：

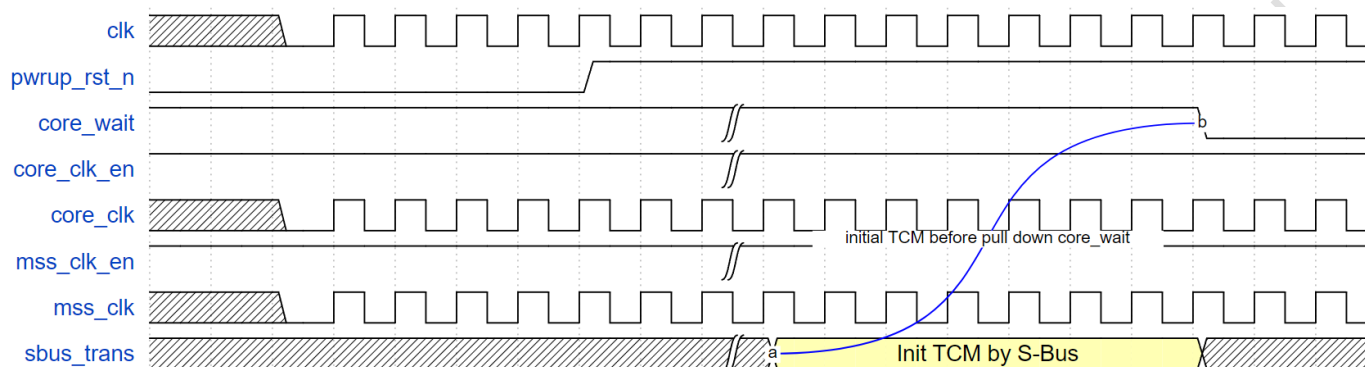


Figure 4-3 Wing-M130 Boot From TCM 时序图(2)

具体流程描述如下：

步骤 1：SoC 上电，打开 Wing-M130 的主时钟时钟，拉高 `core_wait` 信号，并保持 `pwrup_rst_n` 拉低，此时 Wing-M130 处于复位状态。（这个方式不需要使用 `core_clk_en` 和 `mss_clk_en`，可以保持拉高）

步骤 2：在恢复 Wing-M130 的主时钟时钟至少 5 个 cycle 后，拉高 `pwrup_rst_n`，Wing-M130 解复位

步骤 3：拉高 `pwrup_rst_n` 后至少五个 cycle 后，用户可以通过 S-Bus 将初始化程序/数据加载到 TCM 中

步骤 4：完成初始化程序/数据加载后，拉低 `core_wait`，Wing-M130 开始执行

4.2. 静态输入信号

Wing-M130 支持如下静态输入接口，这些接口都要求 SoC 确保，在解除 Wing-M130 复位前保持稳定至少 3 个 cycle，同时在 Wing-M130 运行过程中，保持稳定不变（建议除特殊需求外，集成时将这此信号固接）。具体的静态信号包含：

- `boot_pc`：控制内核的复位启动 PC，要求 2B 对齐
- `core_itsm_base_addr`：控制核内 ITCM 在 Memory Map 上地址范围的基地址，要求 256KB 对齐
- `core_dtcn_base_addr`：控制核内 DTCM 在 Memory Map 上地址范围的基地址，要求 256KB 对齐
- `core_mmr_base_addr`：控制核内 Memory Map 寄存器在 Memory Map 上地址范围的基地址，要求 256KB 对齐
- `hart_id`：核号，会上报到内核 `mhartid` CSR 中，供软件查询
- `rerl_bank_inst_id`：RERI（RAS 错误记录上报模块）的 Bank ID，会上报到对应寄存器中，供软件查询
- `endianness`：大小端指示
- `dbg_authen`：调试模块鉴权通过指示

4.3. 主数据通路接口

4.3.1. Master 类接口

Wing-M130 的 Master 接口主要用于发送核内的 Fetch/Load/Store 以及调试请求到 SoC。

Wing-M130 最多支持配置 3 个 32bit 的 AHB Master 接口，分别命名为 I_BUS, D_BUS, M_BUS。这三个接口均遵循 AMBA 5 AHB Protocol Specification Issue C 定义（需要注意，非完整支持，仅支持的部分遵循相关协议定义）。主要特性如下：

- 支持 32bit 地址总线+32bit 数据总线
- 仅支持 IDLE 和 NONSEQ 类型传输
- 仅支持 Single 传输，不支持 Burst 传输
- 支持基于 PROT 接口传输如下 memory 属性信息
 - 指令/数据访问，其中 IFU 发出的取指请求携带指令访问标志，LSU 或 Debug 发出的 Load/Store 请求携带数据访问标志。
 - 特权模式，当 Wing-M130 内核处于 Machine Mode 下发出的请求携带 Privileged 标志，处于 User Mode 下发出的请求携带 Unprivileged 标志
 - Bufferable 属性，基于 PMA 查询得到
 - Cacheable 属性，基于 PMA 查询得到
- 默认不支持 Exclusive 传输相关接口

需要注意：Wing-M130 提供了基于每个 Master 接口的插拍配置，用户也可以在 SoC 中进行插拍处理，但是需要意识到，针对 Master 接口的插拍会导致内核的性能表现下降。默认配置下 M130/M132 版本的对外接口没有插拍，M134/M136 版本的对外接口有输出方向的插拍

4.3.2. Slave 类接口

Wing-M130 的 Slave 接口主要用于 SoC 访问核内的 Memory Map 空间（包括寄存器和 TCM）。

Wing-M130 支持配置 1 个 32bit 的 AHB Master 接口，S_BUS。这三个接口均遵循 AMBA 5 AHB Protocol Specification Issue C 定义（需要注意，非完整支持，仅支持的部分遵循相关协议定义）。主要特性如下：

- 支持 32bit 地址总线+32bit 数据总线
- 仅支持 IDLE 和 NONSEQ 类型传输
- 仅支持 Single 传输，不支持 Burst 传输
- 支持 Prot 接口，但 Wing-M130 不会使用这个信息
- 不支持 Exclusive 传输相关接口
- 不支持 Mastlock 接口

需要注意: Wing-M130 的 Slave 接口在访问内部资源时, 默认优先级是低于内核访问的, 因此 SoC 需要支持 S-Bus 可能的反压行为。

4.3.3.核内请求与 Master 接口路由关系

Wing-M130 支持配置将内核中的 Fetch/Load/Store 三类请求与对外接口的路由关系, 例如禁止 Fetch 请求发送到 D_BUS。

如果 Fetch/Load/Store 发出的请求落在不支持路由的接口, 则会触发 Access Fault, Wing-M130 将 Trap 到对应的异常服务程序。

Fetch 请求固定无法路由到 P_BUS 和内部寄存器空间 (不可配置), 其余路由关系均可配置开关。

相关配置可以在 m130_bus_cfg_define.sv 中查询与配置

4.3.4.Master 接口空间划分

Wing-M130 支持配置每个 Master 接口对应的地址空间

在 wing_m130_bus_cfg.sv 文件中可以查询/配置对应的 BUS 的地址空间, 每个 Master 接口都通过一组 START_ADDR 和 END_ADDR 参数控制其对应的地址空间范围 (闭集)。

Wing-M130 默认的 Master 接口空间划分可以参考 4.4.1 系统 Memory Map 划分章节。用户也可以基于实际需要配置每个接口对应的地址空间范围, 但是在配置时有如下注意点:

- 建议所有 Master 空间叠加在一起, 可以覆盖所有 32bit 地址空间, 如果无法完整覆盖, 建议打开 DUMMY_SLAVE 配置项, 或者通过 PMA/PMA 配置将未覆盖空间配置为不可访问状态 (例如 nXnRnW 属性), 否则可能存在挂死风险。

特别注意: 如果不通过 PMP/PMA 属性约束, Wing-M130 可能会发出 (地址不确定的) 投机访问到子系统导致挂死, 因此无法完全从软件层面约束。

- 建议每个 Master 空间不重叠, 如果有需要配置多个 Master 之间存在重叠的地址, 则需要基于上一个子章节中描述的方式, 约束 Fetch/Load/Store 请求发往有重叠空间的 Master 的路由通路, 保证三个请求源头的请求, 访问某一个地址只有一条路由通路。例如 I_BUS 与 D_BUS 有地址重叠, 在需要约束 Fetch 只能路由到 I_BUS, Load/Store 只能路由到 D_BUS。如果违反该配置约束, Wing-M130 无法正常工作。

4.4. Memory Map 空间介绍

4.4.1.系统 Memory Map 空间介绍

Wing-M130 支持基于配置的 Master 接口划分系统 Memory Map 空间, 默认配置如下:

				0xFFFF_FFFF	
M_BUS			Reserved	0xF000_0000	Non-Cacheable+Non-Idempotent +RWnX
		PPB(256MB)	PPB	0xE000_0000	
			Reserved	0x6000_0000	
			Reserved	0x4600_0000	
		Device(0.5GB)	Device Bit Band	0x4200_0000	
			Device	0x4000_0000	
		SRAM(0.5GB)	Reserved	0x2600_0000	Cacheable+Idempotent +RWX
			SRAM BITBAND	0x2200_0000	
			SRAM	0x2000_0000	
I_BUS	D_BUS	CODE(0.5GB)	NVM	0x0000_0000	
Non-Idempotent	IO device属性, 不可取指				
Idempotent	memory属性, 可进行取指和数据访问				

Figure 4-4 系统 Memory Map 空间划分

此配置下, 整个 Memory Map 空间被分为 3 段空间

- 低 512MB 空间默认为指令空间, Wing-M130 内核可以通过 I_BUS/D_BUS 访问这段空间。
- 中间 512MB 空间默认为 Main Memory 空间, Wing-M130 内核可以通过 M_BUS 访问这段空间
- 高 3GMB 空间默认为 Device 寄存器和外设寄存器空间, Wing-M130 内核可以通过 M_BUS 访问这段空间

注意: 上述空间划分均可以通过配置修改

4.4.2.Wing-M130 核内 Memory Map 空间介绍

Wing-M130 内核默认的 Memory Map 空间定义如下:

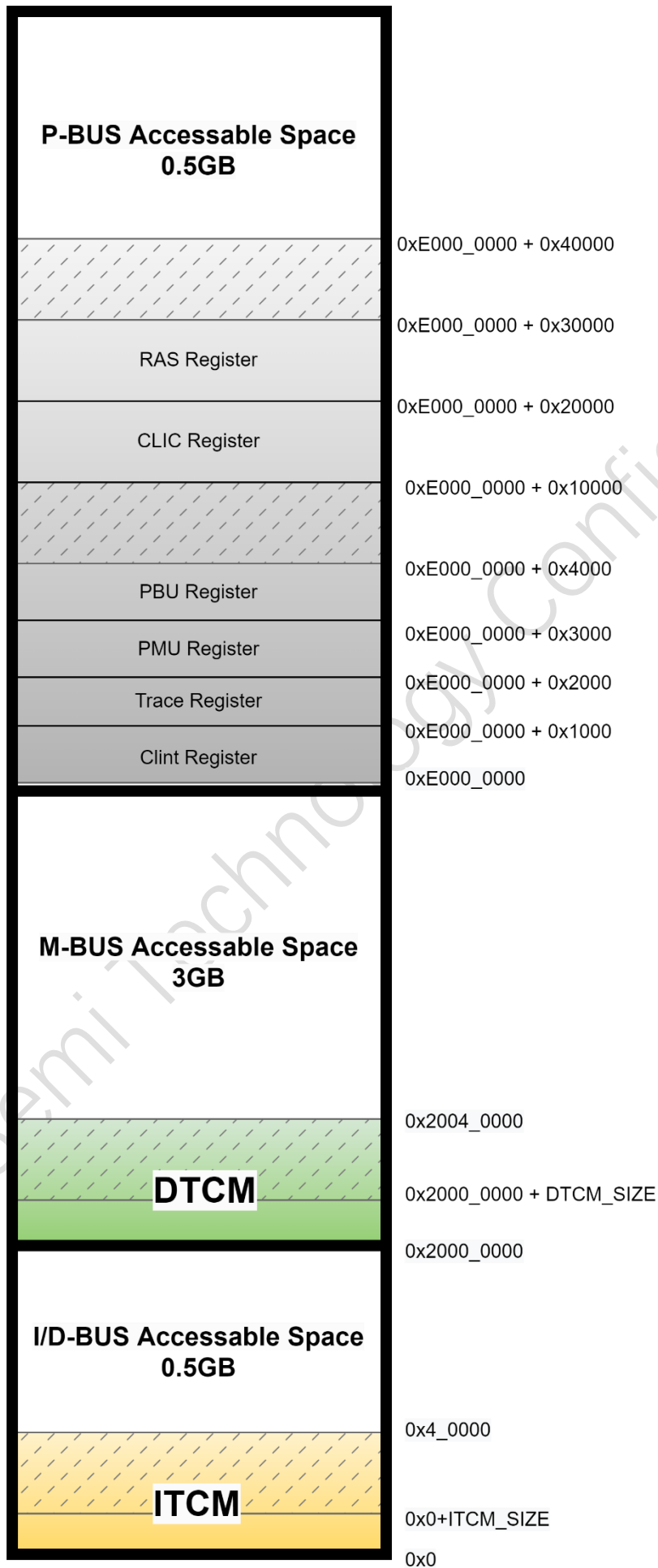


Figure 4-5 M130 Local Memory Map

上图中将 Wing-M130 内部的 Memory Map 空间结合整个空间进行了展示，其中着色的部分为 Wing-M130 内部的 Memory Map 空间，主要为：

- 寄存器空间，占据 256MB 空间，包含 Wing-M130 中 CLIC/Trace/RERI 等模块的寄存器空间，具体每个模块寄存器的基地址偏移参考上图。寄存器空间中同样包含保留空间，如果 Wing-M130 内核试图访问 DTCM 的保留空间，会导致 access fault 异常，如果 SoC 通过 S_Bus 试图访问寄存器的保留空间，会返回一个错误响应。

*需要注意：*上图展示的核内空间基地址为默认配置，这三段核内空间的基地址偏移可以通过宏定义配置，用户可以根据实际需要进行配置。但是需要保证配置出来的三段空间没有重叠。同时，Wing-M130 的 Memory Map 空间是落在全局 Memory Map 空间的，SoC 需要确认外部请求访问这些空间的路由正确性。同时对于 Wing-M130 发出的请求，会优先将这部分地址视作核内空间访问，不会发出到 SoC。

4.5.PMA 配置说明

PMA(Physical Memory Attribute)为 RISC-V 定义的 Memory 属性，与 SoC 平台绑定，不可动态配置（没有对应的 CSR/MMR）。由于 Wing-M130 无法假设 SoC 的形态，因此提供了一套静态配置方式，允许用户结合 SoC 实际形态，配置对应的 PMA 属性。

用户可以在 `m130_pma.svh` 文件中定义 PMA 属性，每个 PMA Entry 包含如下配置项：

- Mode: 0 表示 TOR 模式，表示这个 Entry 的地址范围由两个任意的起始和终止地址定义；1 表示 NAPOT 模式，表示这个 Entry 的地址范围是一个 2 的整数次幂对齐的起始地址+Size 定义；
- Addr_base: 表示起始地址
- Addr_end: 表示终止地址，仅当 Mode 配置为 TOR 模式时使用
- Addr_size: 表示一个对齐空间的对齐 size，例如 10 表示 $2^{10} = 1024B$ 对齐空间，仅当 Mode 配置为 NAPOT 模式时使用
- Cacheable: 表示这个 entry 空间的 cacheable 属性
- Idempotency_n: 表示这个 entry 空间的幂等属性，配置为 1 表示非幂等，当一个空间配置为非幂等时，Wing-M130 不会对齐进行投机的访问，适用于访问有 side-effect 空间。
- R/W/X: 表示这个 entry 的读/写/执行权限，每 bit 独立设置

以下为两个 PMA Entry 的配置实例：

```
{mode:1'b0, addr_base:32'h0010_0000, addr_end:32'h1FFF_FFFF, addr_size:5'd29, cacheable: 1'b1, idempotency_n: 1'b0, vacant: 1'b0,r:1'b1,w:1'b1,x:1'b1},
{mode:1'b1, addr_base:32'h0008_0000, addr_end:32'h000F_FFFF, addr_size:5'd19, cacheable: 1'b0, idempotency_n: 1'b1, vacant: 1'b0,r:1'b1,w:1'b1,x:1'b0},
```

对应的配置意图为：

Entry0 为地址 `0x8_0000` 开始的 512MB(2^{19})的对齐空间，Non-Cacheable+Non_idempotency 属性，可读可写不可执行

Entry1 为地址 `0x10_0000~0x1FFF_FFFF` 空间，Cacheable+idempotency 属性，可读可写可执行

需要注意：

1. 用户可以在配置中关闭 PMA，上述配置仅在 PMA 开启时生效，如果配置关闭 PMA，所有空间的属性都默认为 Cacheable & Idempotency。
2. PMA entry 的定义，不允许出现地址空间的重叠。
3. 当 Wing-M130 的请求不满足 PMA 的属性定义，会触发 access fault 异常。

4.6. 存储子系统集成说明

4.6.1.TCM 集成说明

Wing-M130 支持配置 ITCM/DTCM，各支持一个访问口，默认支持 fetch/load/store 访问 ITCM/DTCM，建议客户将程序放在 ITCM 中，数据放在 DTCM 中，减少访问冲突导致的性能下降。ITCM 的 memory bank 固定为 72bit 位宽，DTCM 的 memory bank 固定为 39bit 位宽

ITCM 和 DTCM 在 memory map 中的地址划分可以参考 4.4.2 章节。

ITCM 和 DTCM 可以已通过内核采取 load/store 方式初始化，也允许 SoC 通过 S-Bus 初始化。

4.6.2.ICache 集成说明

Wing-M130 支持配置 ICache，ICache 默认采取两路组相连的实现，容量支持 4KB~64KB (Power-of-two)。取指操作访问 Cacheable 空间 (PMA 中定义) 时，可以通过 ICache 进行加速。

需要注意，当同时配置了 TCM 时，TCM 中的指令数据不会进入 ICache 中。

ICache 需要例化 tag memory 和 data memory，所例化的 memory bank 数量与组相连规格绑定，两路组相连时支持 2 个 tag memory bank 和 2 个 data memory bank，分别对应 way0 与 way1。

解复位后，Wing-M130 会自动对 ICache 进行初始化，初始化需要持续一段 cycle，初始化完成后内核才会启动执行。软件也可以通过 Fence.i 主动清空 ICache。

ICache 支持软件通过内部的 MMR 配置使能，解复位后默认不使能 ICache，需要软件通过写 iccen.en 位域 1 来使能 ICache。

4.6.3.Memory Wrapper 集成说明

Wing-M130 默认例化的 memory wrapper 在外部，Wing-M130 通过固定的一组接口与 memory wrapper 互联。

系统进行 memory 选项时，需要注意：

- 需要支持 bit enable 功能。
- Memory 的 DFT, Low Power 等接口，在外部集成时控制，不需要通过 Wing-M130

4.6.4.Memory ECC 实现说明

当配置 TCM 或 ICache 时，相关的 memory 支持 ECC 保护（memory 不在锁步范围内）。

对于 TCM 的 ECC 保护，支持 1bit 错误硬件自动纠错，以及 2bit 错误上报，并记录相关错误信息到 RERI 模块。

对于 ICache 的 ECC 保护，支持 1bit/2bit 错误均采用 invalid+refill 出错的 cacheline 方式纠错，并记录相关错误信息到 RERI 模块。

需要注意，由于 DTCM 的 ECC 保护为 32bit，因此非 word 操作的写（Byte or Half Byte）均会导致硬件触发读改写的操作，需要额外的 cycle 完成，对性能会产生一定影响。

注：开启 Trace 功能后，Trace 包含一块 Trace Memory，Trace 属于调试功能，因此固定不支持针对 Trace Memory 的 ECC 功能

4.7. 中断系统集成说明

Wing-M130 支持配置两种中断控制器，CLIC 与 CLINT。其中 CLINT 仅包含最基本的外部中断/软件中断/定时中断各 1 个，以及可选的 NMI 中断。而 CLIC 在 CLINT 的基础上，增加了更多外部中断线的支持，并且支持更丰富中断优先级，中断嵌套，中断咬尾等功能，用户可以根据实际需要，选择配置使用哪种中断控制器。

4.7.1.外部中断集成说明

CLINT 模式下的外部中断通过 ext_int 信号输入，CLIC 模式下外部中断保留了此信号输入，同时还额外扩展了 N bit loc_int 输入线。

注意：

1. SoC 输入的外部中断线，默认为同步输入，如果需要进行异步处理，需要将 Wing-M130 中相关宏打开
2. 用户如果选择 CLIC 模式，可以根据实际情况选择是否使用 ext_int 中断线，Wing-M130 在 CLIC 模式下保留 ext_int 中断线，目的为兼容 CLINT 模式，此中断输入与 loc_int 中断输入并没有区别（除中断号有区别外）。因此客户如果只想集成一组中断信号，则可以选择将 ext_int 固接 0，不使用此中断输入。

4.7.2. 软件中断集成说明

Wing-M130 支持的软件中断，是通过写内核中一个指定的 MMR 地址触发，这个操作可以由 Wing-M130 写触发，也可以由 SoC 中其他组件的写触发。

注意：如果使用 CLIC 模式，建议用户将软件中断固定为电平触发中断

4.7.3. 定时中断集成说明

Wing-M130 支持基于参考时钟产生的时间戳与配置门限比较产生的定时中断。其中时间戳有两种方式得到，通过 EMBEDDED_TIMER 宏控制。

当这个宏关闭时，需要 SoC 输入基于参考时钟的时间戳，SoC 需要负责确认此时间戳的精度，以得到准确的定时中断效果。

当这个宏开启时，需要 SoC 输入一个稳定的参考时钟，Wing-M130 会基于此时钟进行计数产生时间戳信息，用于产生 EMBEDDED_TIMER 宏。SoC 需要确认输入的参考时钟不会被动态调频率或者关断（也即基于一个稳定的频率给到 Wing-M130）

注意：如果使用 CLIC 模式，建议用户将定时中断固定为电平触发中断

4.7.4. NMI 集成说明

Wing-M130 支持可选的 NMI 中断输入，并且支持配置为 RNMI，两者的区别是 NMI 中断响应是否会破坏内核现场状态，是否可以从 NMI 中正确返回。用户可以根据实际需要选择配置是否需要 NMI 中，以及是否需要可恢复的 NMI

4.7.5. 中断属性配置说明

当中断控制器配置为 CLIC 时，支持一个中断属性静态配置文件，如果用户使用的系统中部分中断属性为固定值，则用户可以通过此文件，将指定中断的指定属性固接（取代正常实现下的配置寄存器），从而减小中断控制器的面积和功耗开销。

配置文件位于 include/wing_clic_mmr_config.svh，具体格式和使用说明如下：

```
localparam clic_mmr_cfg_s [`CLIC_EXT_INT N-1 : 0] CLIC_UNIT_MMR_CFG = {
  `CLIC_EXT_INT_N{3'b000, {`CLIC_INTCTLBITS_W{1'b0}}, 2'b00, 1'b1}
};
```

注：上述代码表示的所有 CLIC 中断都不采取固接的方式，如果客户不需要固接相关属性，则可以采取上述默认配置

用户需要按照实际使用场景，填写这个文件中的 clic_mmr_cfg_s，其中每一行代表对应中断 ID 的固接方式，四个值分别代表：

- 对应后续三个域{中断优先级，中断触发模式，中断使能}是否采取固接值，1 表示固接，0 表示不固接
- 中断优先级的固接值，值越大优先级越高，需要注意有效的取指范围应该参考 CTLBITS 参数的值，例如 CTLBITS 配置为 4 时，只支持 16 个中断优先级(0~15)
- 中断触发模式的固接值，其中 bit0 定义脉冲中断/电平中断，0 表示电平中断，1 表示脉冲中断；bit1 定义触发方式，0 表示低电平/下降沿触发，1 表示高电平/上升沿触发
- 中断使能的固接值，0 表示固接不使能，1 表示固接使能

4.8. 调试系统集成说明

Wing-M130 支持基于 IEEE 1149.1 标准的五线 JTAG 接口，或者基于 IEEE 1149.7 标准的两线 CJTAG 接口。

当配置为两线 CJTAG 口时，Wing-M130 会输出 tms（输入），tdo（输出），tdo_en（输出）信号，SoC 需要负责将这三个信号，使用一个三态门，产生一个 inout 类型信号，与标准两线 CJTAG 接口对接。

4.8.1. 调试接口跨时钟域说明

JTAG/cJTAG 接口采用的时钟域与 Wing-M130 的主时钟域为异步关系。涉及跨时钟域的处理基本集中在 wing_m130_top.u_m130_ds_top.m130_dtm_tapc_synchronizer 层次下，具体如下：

- 从 tck 时钟域到 clk 时钟域，通过生成 tck 时钟产生二分频信号，并将上升沿/下降沿信号在 clk 时钟域打两拍的方式，生成采样 valid 信号，对相关信号采样，完成跨时钟域处理，包含如下信号：
 - tapc2tapcsync_dmi_ch_sel_i
 - tapc2tapcsync_ch_id_i
 - tapc2tapcsync_ch_capture_i
 - tapc2tapcsync_ch_shift_i
 - tapc2tapcsync_ch_updata_i
 - tapc2tapcsync_ch_tdi_i
 - tapc2tapcsync_ch_tdo_i
- 从 clk 时钟域到 tck 时钟域，所有输出回 tck 时钟域的信号，都是基于上述由 tck 时钟域信号跨到 clk 时钟域信号控制生成，并直接返回给 tck 时钟域，tck 与 clk 的时钟周期主频倍数差保证在下一个 tck 时钟沿可以正确的采到返回的信号，包含如下信号：
 - tapcsyn2core_ch_id_o
 - tapcsyn2core_ch_capture_o
 - tapcsyn2core_ch_shift_o
 - tapcsyn2core_ch_update_o
 - tapcsyn2core_ch_tdi_o

4.9. 低功耗集成说明

Wing-M130 处于一个 power domain 中，并且支持基于 WFI 指令的低功耗模式。

4.9.1. WFI 低功耗流程

当 Wing-M130 执行 WFI 指令后，内核将进入 WFI 低功耗模式，具体流程如下：

步骤 1：内核执行 WFI 指令，停止内核流水线执行

步骤 2：等待 IFU/LSU 等模块进入 Idle（具体是等待在途的总线请求完成），然后 Wing-M130 将会通过 sleeping/deep sleeping 状态指示 SoC（也可以直接使用 clk_gate_en），SoC 可以基于此信号对输入 Wing-M130 的 clk 进行门控关断。

在 WFI 低功耗模式下，只有 M130 内核处于时钟关断状态，包括中断，调试在内的逻辑的时钟都会保持开启，因此：

➤ 外部中断/时钟中断/软件中断将依然被采样和触发，并上报给 Wing-M130 内核

当 Wing-M130 检测到如下事件后，将会推出 WFI 模式并提交掉 WFI 指令：

1. 复位
2. 收到 DM 的调试请求
3. 收到 CLIC 上报的（可以响应的）中断

需要注意：上述提到的中断触发唤醒 WFI 低功耗模式，对应的中断应该满足其触发条件，如下：

1. 对于 CLINT 模式，如果中断对应的 CSR MIE.msie/meie/mtie 位域配置为 0，此时对应的中断无法被内核采样，也因此不会从 WFI 低功耗状态下唤醒。但是 CSR MSTATUS.mie 域的配置不会影响内核采样中断和退出 WFI 低功耗模式
2. 对于 CLIC 模式，是类似的机制，每个中断都需要遵循其采样的条件来判断是否唤醒 WFI 低功耗模式

需要注意：

1. Wing-M130 采样到中断退出 WFI 低功耗模式，和退出低功耗模式后是否进入中断是独立判断的，并非绑定的行为
2. 用户在使用时，需要确保进入 WFI 低功耗模式前，正确配置中断相关属性，防止中断无法唤醒 WFI 低功耗模式

内核进入和退出低功耗模式的时序示意图如下：

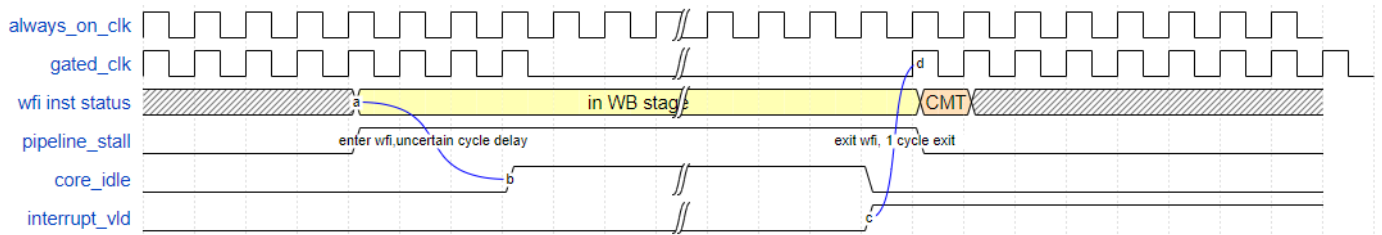


Figure 4-6 M130 WFI 低功耗模式时序图

4.10. DFT 集成说明

Wing-M130 对于内部例化的 ICG cell，以及内部生成的复位信号，均支持基于输入的 DFT 接口可控。需要注意，SoC 需要额外针对输入的复位信号，进行 DFT 可控设计 (DFT MUX)。

5. 后端实现指导

5.1. 接口信号时序约束建议

参考 T28 ssg0p81vm40c 工艺下，200MHz 主频，建议输入输出按照 40%进行过约

需要对于 TCK 与 CLK 跨时钟域处理部分，需要额外增加如下 max_delay 不超过 50%clk 时钟周期的综合约束：

- From *wing_m130_top.u_m130_ds_top.i_dtm_top.i_tapc_synchronizer.tapcsync2dmi_ch_sel_o* to *wing_m130_top.u_m130_ds_top.i_dtm_top.i_tapc.tdo_out_next*
- From *wing_m130_top.u_m130_ds_top.i_dtm_top.i_tap.i_tap_idcode_reg.shift_reg[0]* to *wing_m130_top.u_m130_ds_top.i_dtm_top.i_tapc.tdo_out_next*
- From *wing_m130_top.u_m130_ds_top.i_dtm_top.i_dmi.tap_dr_ff[0]* to *wing_m130_top.u_m130_ds_top.i_dtm_top.i_tapc.tdo_out_next*

5.2. 版图规划建议

暂无

6. Wing-M130 HDK

Wing-M130 HDK (Hardware Development Kit) 是一个硬件集成开发环境，为用户提供了基于 Wing-M130 内核，预置的编译/仿真/综合/LINT 检查等功能，方便客户快速评估和熟悉 Wing-M130，具体包括：

- ✓ 基于提供的测试集的仿真环境
- ✓ 基于 JTAG+OpenOCD 的调试仿真环境
- ✓ RTL Lint 检查
- ✓ DC 综合评估

6.1. Wing-M130 directory

整个 Wing-M130 HDK 目录结构如下：

Table 11 Wing-M130 HDK 目录结构

Folder	Description
CFG.mk	Basic ISA & Arch configuration for this database.
Makefile	Make file script to run database
README.md	Simple database run guide
build	Dependent configuration
---env.sh	Environment variables setting file, user must source this file before running this database
doc	Document
---Wing-M130_Core_TRM.pdf	Wing-M130 CORE Technical Reference Manual
---Wing-M130_Core_IM.pdf	Wing-M130 CORE Integration Manual
hardware	HDK submodules
---design	Design directory, include RTL and file list
---demo_test	RTL simulation environment for provided basic test
---fpga	FPGA Project
---spyglass_lint	SPYGLASS LINT check environment
---syn_dc	DesignCompiler Synthesis environment
software	SDK submodules
---host	Software executable files, include GCC and openocd
---target	Simulation program (Hello, Dhrystone, Coremark)

Wing-M130 的源码 filelist 可以在 `./hardware/design/filelist` 目录下找到，包含两个 list：

- **wing_m130_top.list** - Wing-M130 的仿真 filelist
- **wing_m130_top_syn.list** – Wing-M130 的综合 fielist

6.2.HDK 使用说明

6.2.1.Database 设置

在用户开始使用/评估此 Database 前，应该需要先在根目录下执行 `source ./build/env.sh`

重要说明

此数据库是基于 csh 环境建立的，如果客户的在 bash 或其他环境下评估，需要手动适配一下 `./build/env.sh`

6.2.2.工具链依赖检查

此数据库的评估依赖一些 EDA 工具链，因此在开始评估前，需要先执行 `make check` 命令

这个命令会在 terminal 上展示工具链的检查结果，如果有部分工具链缺失，则对应的一些检查项可能无法运行，具体每个检查项依赖的 EDA 工具如下：

Table 12 工具链依赖

Estimation Goals	Dependency EDA/Tool Chain
Simulation	(Cadence) XRUN or (Synopsys) VCS (Wing) GCC
Debug	(Cadence) XRUN or (Synopsys) VCS (Wing) OpenOCD
Lint	(Synopsys) Spyglass
Synthesis	(Synopsys) DesignCompiler
Formality	(Synopsys) Formality

对于可以选择多个 EDA 执行的目标，用户可以通过命令指定其中一个来评估，具体方式见下文。

其中由隼瞻公司提供的工具链可以在 `./software/host/bin/gcc` 中找到

6.2.3.仿真环境说明

6.2.3.1. Built-in 测试

整个仿真环境的测试用例放在在 `./software/targets/benchmark` 目录下，包含了如下内建的测试用例：

- **Helloworld** - "Hello" sample program
- **Dhrystone** - Dhrystone benchmark
- **Coremark** – Coremark benchmark

6.2.3.2. Testbench 描述

Wing-M130 testbench 包含了 Wing-M130 核以及相关的环境组件，这套 Testbench 提供了一套机制，可以将程序中打印行为，写到某个具体的地址，从而打印到用户的 terminal 上。同时也提供了一个特殊的结束 PC，当程序运行到这个 PC 时，会自动退出仿真。对应如下：

Table 13 Wing-M130 仿真控制地址

Address	Description
0XF0000000	Print Character
0X000000F4	Simulation Exit

6.2.3.3. 执行仿真

默认情况下，用户可以直接使用如下命令，不带任何额外参数，来执行仿真

Make Run

这个命令将会运行所有预置的测试用例

如果客户希望跑某个特定的测试用例，则可以使用 TARGETS 选项来指定，如下：

Make TARGETS="helloworld" - 指定运行 helloworld

Make TARGETS="dhrystone coremark" - 指定运行 dhrystone 和 coremark

同时 HDK 还提供其他选项来进一步控制仿真的执行，具体如下：

- **SIMULATOR=xrun**，用于指定仿真使用的 EDA 工具，有效的参数为“xrun”“vcs”（不指定默认为使用 xrun）
- **SIM_BUILD_OPTS="+define+DUMP_FSDB"**，用于指定是否 dump 波形文件（不指定默认不 dump 波形）

以下为几个仿真命令示例：

Make TARGETS="helloworld" SIMULATOR=vcs SIM_BUILD_OPTS="+define+DUMP_FSDB"

Make TARGETS="helloworld coremark" SIMULATOR=xrun

在仿真结束后，在 HDK 中会创建两个零时目录，来存放仿真的过程文件和结果。第一个目录在 `./hardware/demo_test/workload`，其中保存的测试用例的编译结果（elf 文件）。另一个目录在 `./hardware/demo_test/run`，其中保存仿真的结果文件（log，波形等）。

6.2.3.4. 仿真结果确认

用户可以执行如下命令来打开仿真 log 文件:

Make [testcase_name]_log

例如, 用户想要打开 Dhrystone 的仿真 log 文件, 则命令为:

Make dhrystone_log

如果用户希望进一步打开波形文件, 则需要在仿真时加上 **SIM_BUILD_OPTS="+define+DUMP_FSDB"**, 则波形文件可以在 `./hardware/demo_test/run/test.fsdb` 目录下找到, 用户可以使用自己的波形调试工具进行调试。

6.2.4. Lint 检查环境说明

用户可以执行如下命令, 使用 Spyglass 命令进行 Lint 检查:

Make run_spyglass

当执行这个命令后, 会使用 Spyglass 的 batch 模式进行 Lint 检查, 当检查结束后, 会自动打开图像界面, 方便用户进一步确认检查结果

6.2.5. 综合环境使用说明

6.2.5.1. DB & SDC 设置

在用户开始综合评估之前, 应该先在如下文件中设置相关的库文件

`./hardware/syn_dc/scr/db.tcl`

这个文件中包含一些 demo 设置, 但是不一定适配用户的需要, 因此用户可以将本次评估实际需要的标准单元 /Memory 工艺库文件, 参考 demo 的写法, 放入其中。

然后用户需要在 SDC 中确认/修改本次评估的时序约束:

`./hardware/syn_dc/input/top.sdc`

6.2.5.2. 综合评估

用户可以用如下命令, 使用 Design Compiler 进行综合评估:

Make run_dc [DC_FREQ=xxx]

其中可选的参数 **DC_FREQ=xxx** 可以指定本次评估的主频，单位为 MHZ，如果不指定，则会使用 SDC 文件中默认的主频

6.2.5.3. 综合结果确认

当用户完成综合后，综合报告可以在如下目录获取到：

```
./hardware/demo_test/run/dc_log/dc_[timestamp]
```

其中[timestamp]后缀为启动综合的时刻点

6.2.6.JTAG 仿真环境使用说明

此 HDK 提供了 JTAG 仿真环境，用户需要按照如下步骤启动：

Step 1. 执行 **make run_jtag**

Step 2. 开启一个新的 terminal 并执行 **make run_openocd**

Step 3. 再开启一个新的 terminal 并执行 **make run_gdb**

Step 4. 在步骤 3 中的 terminal 中输入 '**tar ext :3333**'

正确完成上述步骤后，就可以将 GDB 挂载到仿真环境中，此时可以通过 GDB 命令开始调试 Wing-M130 内核，并且调试过程操作都会 dump 到对应波形中供调试（如果开启了 Dump 波形选项）

6.2.7.Database 清除

用户可以执行如下命令来清除之前评估过程中的临时文件：

Make Clean